Chapter 1

Control Architectures

Chapter **??** defined the residual wrench and the two ways of obtaining it. This chapter describes the control strategies compared in this thesis, and in each case the point at which knowledge of the interaction enters the control law.

Four controllers: an uncompensated baseline (Strategy 0), flown in every scenario as the reference condition, and the three strategies compared against it (Strategies 1–3). Table 1.1 lists them, and the rest of the chapter takes them in order.

The chapter treats each strategy as its own controller. What makes the comparison fair is not a claimed shared implementation, but the experimental conditions held constant for every one of them — the same airframe, trajectories, formations, separations, residual definition and gains frozen per strategy after tuning, stated precisely in Section 1.6.

Each strategy is described in two parts, kept deliberately separate: what is implemented here, and the published method it follows. They follow published *families*, and the deviations from each source paper are listed in that strategy's section — none is a bit-identical replica. Strategies whose author code has not yet arrived stay unimplemented rather than guessed.

1.1 The position and attitude loop

Strategies 0, 1 and 2 (the geometric baseline, pure INDI and geometric+NN) each use a geometric position loop and an attitude law realising the corresponding thrust direction; Strategy 1 replaces the attitude law's inner term with an incremental one. This is stated once here, in its own section, purely to avoid repeating the same equation three times. Strategy 3 uses a different published structure entirely, feedback linearisation, described in its own section instead.

1.1.1 Position loop

The desired acceleration is

$$a_{\text{des}} = \underbrace{a_{\text{ff}}}_{\text{trajectory}} + K_p e_p + K_v e_v + K_i \int e_p \, \mathrm{d}t + g e_3 \underbrace{- a_{\text{res}}}_{\text{measured}} \underbrace{- \hat{a}_{\text{res}}}_{\text{predicted}}, \tag{1.1}$$

with $e_p = p_{\text{des}} - p$ and $e_v = v_{\text{des}} - v$, and gains diagonal but distinct in the horizontal and vertical axes. The commanded thrust vector is $m\, a_{\text{des}}$; its magnitude projected onto the body z-axis gives the collective thrust, and its direction, together with the desired yaw, gives the desired attitude R_{des} through the usual flatness construction.

Table 1.1: The strategies compared, and the implementation status of each at the time of writing. "Knows" refers to the residual: whether the controller has a measurement of it, a prediction of it, both, or neither.

#	Strategy	Knows	Enters at	Rotor speed	Status
0	Geometric baseline	nothing	—	no	flying
1	Pure INDI	measured	a_{des}, torque	yes	flying
2	Geometric + NN	predicted	a_{des}	no	implemented, unflown
3	FBL + NN	predicted	linearising law	no	not implemented

The last two terms of (1.1) are the whole subject of this thesis, and both are subtracted, for the reason derived in Section **??**. *Which residual information is present is what distinguishes Strategies 0, 1 and 2 from one another*, and nothing else about the position loop changes between them: Strategy 0 has neither term, Strategy 1 the measured one, and Strategy 2 the predicted one. Strategy 3 injects its prediction elsewhere, in the feedback-linearising law of Section 1.5.

1.1.2 Attitude loop

The attitude error is the standard geometric one, $e_R = \frac{1}{2}\left(R_{\text{des}}^\top R - R^\top R_{\text{des}}\right)^\vee$, with $e_\omega = \omega - \omega_{\text{des}}$. In the geometric configuration the torque is

$$\tau = -K_R e_R - K_\omega e_\omega + \omega \times J\omega - k_{i,\text{att}} \int e_R \, \mathrm{d}t, \tag{1.2}$$

the third term compensating the gyroscopic coupling. This is a reduced form of the law in [2]: it drops that work's angular-feedforward bracket and adds an integral term the source does not have — see (1.3) and the discussion in Section 1.2. The INDI configurations replace (1.2) with an incremental law, described in Section 1.3.

1.1.3 Runtime selection

The control law is selected by a parameter at runtime rather than by recompiling: one bit enables the measured residual in the position loop, another selects the incremental attitude law, and a third parameter enables the learned prediction. All are pushed to the vehicle at connection time from a single configuration file.

This matters more than it may appear. Comparing strategies that require different firmware introduces a confound that is impossible to rule out afterwards, a flash cycle changes more than the control law, and strategies flown on different days meet different battery states, different temperatures and different motion-capture calibrations. Runtime selection allows the strategies to be interleaved within a single session, which is what makes the comparison a paired one.

1.2 Strategy 0: uncompensated geometric baseline

Implemented. Equations (1.1) and (1.2) with both residual terms zero. No knowledge of the interaction whatsoever.

Reference. Geometric tracking on $SE(3)$ [2], which tracks a position trajectory together with a single heading direction, four outputs, not a full six-degree-of-freedom pose. The

attitude error of (1.2) is that work's $e_R = \frac{1}{2}(R_d^\top R - R^\top R_d)^\vee$, verified against the source rather than reproduced from memory.

The implemented torque law is a reduced form, and is labelled as such. The law in the source is

$$M = -k_R e_R - k_\Omega e_\Omega + \Omega \times J\Omega - J\left(\hat{\Omega}\, R^\top R_d \Omega_d - R^\top R_d \dot{\Omega}_d\right), \tag{1.3}$$

with $e_\Omega = \Omega - R^\top R_d \Omega_d$. Equation (1.2) retains the proportional, damping and gyroscopic terms but omits the final feedforward bracket, and adds an integral term on attitude error that the source does not contain. It is therefore a *reduced* law in the same family, not the torque law of [2], and no stability result proved there transfers to it unexamined. Restoring the feedforward bracket would be a firmware change rather than a wording change, and is outside the scope of this draft.

What the source proves. Not global asymptotic tracking. With the attitude error function Ψ, exponential stability is established on $\Psi(0) < 1$, initial attitude errors below $90\,^\circ$, and almost-global exponential attractiveness on $1 \leq \Psi < 2$, below $180\,^\circ$. The distinction matters here only in that this thesis claims neither: the baseline is flown as a reference condition, not as a certified controller.

Role. Strategy 0 is the uncompensated baseline of Table 1.1. It is flown in every scenario so that "how much did compensation help" has a denominator, and so that the interaction can be separated from the model error discussed in Section **??**. It is also the controller under which all training data is collected, precisely because it does not compensate: a residual observed under a compensating controller is the residual that survived compensation, which is not what a predictive model should be fitted to.

1.3 Strategy 1: pure INDI

Implemented. The measured residual of (**??**) enters the position loop as $-a_{\text{res}}$ in (1.1). In the attitude loop the geometric torque law is replaced by an incremental one. A reference angular acceleration is formed from the geometric errors,

$$\alpha_{\text{ref}} = \alpha_{\text{des}} - K_R e_R - K_\omega e_\omega, \tag{1.4}$$

where α_{des} is the feedforward angular acceleration obtained from trajectory snap by differential flatness. The achieved angular acceleration α_{meas} is obtained by differentiating the gyro signal and low-pass filtering it, and the torque currently being applied, τ_{cur}, is reconstructed from measured rotor speeds. The command is then the increment

$$\tau = \tau_{\text{cur}} + J\left(\alpha_{\text{ref}} - \alpha_{\text{meas}}\right). \tag{1.5}$$

Because (1.5) corrects relative to what is actually being achieved, it rejects any disturbance appearing in α_{meas} without needing a model of it. The differentiation and filtering are the source of the delay identified in Section **??**; the filter cutoff sets the trade-off between that delay and the noise admitted from the gyro.

The position and attitude increments are independently selectable, giving four configurations: geometric, position-INDI only, attitude-INDI only, and full INDI. Strategy 1 refers to the full configuration, and that is the controller the comparison reports. The

intermediate ones are diagnostic bits used in the two-robot results chapter to attribute an effect to one loop or the other. They are not additional strategies and will not be reported as Strategy 1.

Reference. The incremental formulation with flatness-derived feedforward follows Tal and Karaman [6]; the cascaded arrangement of position and attitude INDI follows Smeur et al. [5].

Deviations. The airframe, gains and filter cutoffs are ours and were identified for this platform. Tal and Karaman obtain rotor speed from optical encoders [6]; this implementation uses the platform's own rotor-speed measurement. No claim is made that this implementation reproduces the tracking performance reported in either work, that would require a like-for-like replication which is not part of this thesis.

Requirements. Rotor-speed sensing, without which τ_{cur} and a_{res} are both unavailable and the method degrades silently (Section **??**). The method needs no training data; the residual is formed online from inertial and rotor-speed measurements.

1.4 Strategy 2: geometric with a learned residual

Implemented. The geometric controller of Section 1.2 with the deep-sets prediction $\hat{a}_{\text{res}}$ of (**??**) subtracted in (1.1). The network is evaluated onboard at the control rate from the relative states of the neighbours, which the flight stack already broadcasts, so the input requires no additional communication. Weights are trained offline on data collected under Strategy 0 and uploaded before flight.

The prediction is computed and logged *whether it is enabled or not*. This is what makes the model's own accuracy measurable independently of its effect on the vehicle: on a flight where the prediction is not driving the controller, predicted and measured residuals are two independent quantities and their comparison is meaningful. Once the prediction is driving the controller, the measured residual is the one that survived compensation and the comparison no longer isolates model quality. Both flights are therefore needed, and Chapter **??** schedules them.

Reference. The approach of predicting the interaction from relative states with a permutation-invariant network and applying it as feedforward follows the Neural-Swarm line [4, 3].

Deviations. These are substantial and are stated plainly. The architecture here is a deep-sets model of our own sizing, 987 parameters, rather than a reproduction of any published network. Neural-Swarm2 applies spectral normalisation to obtain stability and generalisation guarantees [3]; this implementation does not, and the guarantees that follow from it are correspondingly not claimed. Neural-Swarm2 also uses the learned residual in motion planning as well as control [3], whereas here the trajectories are planned in advance and the residual is used for control only. This strategy is a representative of the predictive family, and should be read as such rather than as an evaluation of Neural-Swarm2.

Requirements. Neighbour relative states; training data. No rotor-speed sensing.

1.5 Strategy 3: feedback linearisation with a flatness-preserving residual

Implemented. Not implemented. The controller code was requested from the authors and had not been received at the time of writing. The strategy does not block the others, and its absence is a scope limitation rather than a failure of the comparison (the discussion chapter).

Reference. Hsieh et al. [1], building on the theory of Yang et al. [7]. The distinguishing property is where the prediction enters: not as a feedforward term added to a desired acceleration, but inside a feedback-linearising law whose validity depends on the augmented system remaining differentially flat. A generic learned residual can destroy that flatness; residuals with a lower-triangular structure, a structured correction inside the state derivative, not a free wrench, preserve it, and with it the original flat output [7]. That theory is established in simulation on a planar quadrotor; the hardware demonstration is Hsieh et al.'s.

Its relevance to this comparison is that it is the strongest published counter-argument to treating "predictive compensation" as one method. Hsieh et al.'s abstract reports a 31 % reduction in average tracking error; on hardware the body reports 24 % RMSE for ROM-FBL over nominal FBL and a further 29 % for learned FBL over ROM-FBL, matching the tracking error published by Chee et al. for nonlinear model predictive control, at an order of magnitude less computation in simulation, from under 30 s of training data, and at a 5 ms loop rate obtained *off-board* [1]. Those are the figures a predictive strategy in this class should be measured against, with the caveat that none of them was produced by a controller running on the vehicle.

Requirements. Neighbour relative states; training data. No rotor-speed sensing.

1.6 What makes the comparison fair, and what it does not guarantee

What is held constant. The airframe, the trajectories, the formations, the separations, the residual definition and the estimator. Each strategy is tuned for itself; those gains are then frozen before the comparison flights. This holds uniformly for every strategy — none is exempted or held to a looser standard on the tasks or the measurement pipeline.

What this does not guarantee. Two limitations are worth stating before results are presented rather than after.

Every strategy follows a published *family*, not a specific paper reproduced exactly; the deviations from each source are listed in that strategy's own section. Strategies whose author code has not yet arrived stay unimplemented rather than guessed. Because each strategy is its own controller, "identical conditions" means identical trajectories, formations, hardware and separations — not identical control structure or one gain set imposed on every law. A difference in outcome could in principle be attributable to a controller's own structure rather than to how it uses the interaction-force information, and this is true of every strategy, not a caveat limited to some of them.

Geometric tracking, INDI and feedback linearisation are different laws and are tuned separately. After that tuning, every strategy's gains are frozen for the comparison flights

so a later gap is not mid-campaign retuning. The cost of per-strategy tuning is in scope for RQ4. The discussion chapter reports where a frozen gain appears to have disadvantaged a strategy.

1.7 Summary

Three strategies (1–3: pure INDI, geometric+NN and FBL+NN), plus the uncompensated baseline (Strategy 0) flown as the reference condition under all of them, each its own controller, distinguished by what they know about the interaction force, when they know it, and where that knowledge enters. All three are compared under the same experimental conditions: identical trajectories, formations, hardware and separations, with each strategy's gains frozen after its own tuning, not identical control structure.

At the time of writing, two strategies fly (the baseline and pure INDI), one is implemented and unflown (geometric+NN), and one is not implemented (FBL+NN). Chapter **??** describes the experimental design; the two- and multi-robot results chapters report what was flown.

Bibliography

[1] Pei-An Hsieh, Fengjun Yang, Nikolai Matni, and M. Ani Hsieh. Flatness-preserving residual learning for real-time tight quadrotor formation flight. arXiv:2607.12275, 2026.

[2] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). In *49th IEEE Conference on Decision and Control (CDC)*, pages 5420–5425, 2010. PRIMARY reference for the geometric SE(3) controller (Strategy 0). Tracks position and one heading direction, four outputs. Its torque law is CDC eq. (16) and includes the angular feedforward term; the firmware implements a reduced form of it. Stability is regional (Prop. 2 exponential below 90 degrees; Prop. 3 almost-global attractiveness below 180 degrees), not global. Its companion is arXiv:1003.2005v1. Do NOT attribute the three flight modes, velocity mode, heading-projection proposition or the switched 720/360-degree example to this paper; those are in the v4 companion only.

[3] Guanya Shi, Wolfgang Hönig, Xichen Shi, Yisong Yue, and Soon-Jo Chung. Neural-swarm2: Planning and control of heterogeneous multirotor swarms using learned interactions. arXiv:2012.05457, 2020. Published in IEEE Transactions on Robotics.

[4] Guanya Shi, Wolfgang Hönig, Yisong Yue, and Soon-Jo Chung. Neural-swarm: Decentralized close-proximity multirotor control using learned interactions. arXiv:2003.02992, 2020.

[5] Ewoud J. J. Smeur, Guido C. H. E. de Croon, and Qiping Chu. Cascaded incremental nonlinear dynamic inversion control for mav disturbance rejection. arXiv:1701.07254, 2017.

[6] Ezra Tal and Sertac Karaman. Accurate tracking of aggressive quadrotor trajectories using incremental nonlinear dynamic inversion and differential flatness. arXiv:1809.04048, 2018.

[7] Fengjun Yang, Jake Welde, and Nikolai Matni. Learning flatness-preserving residuals for pure-feedback systems. arXiv:2504.04324, 2025.